reflex-layers

Table-of-contents

Textures

- Textures are specified in `/assets/data/data.json`
- Texture files need to be placed inside the `assets` folder as angular can only access files in this folder
- Textures are grouped into `TextureResources` which contain the data for a specific use case / scenario.

Texture Formats

1. `Texture2D`
   - multiple images in common file formats (preferably `png` and `jpg`)
   - constraint: maximum 15 images can be used
2. `TextureArray`:
   - Number of images of equal size provided ina single file
   - either in RAW format (raw bit data) or in `Khronos KTX` texture format, could contain 256 or more images
3. `Texture3D`:
   - number of images of equal size, encoded as volumetric data structure
   - either in `DDS` format (raw bit data) or in `Khronos KTX` texture format
   - **REMARK:** currently not working / not implemented as `three.js` *can* read and load DDS textures, but binding to a `Texture3d` is not working (it seems that three.js focuses on Texture Arrays)

Common Properties for TextureResource

- `id`: unique identifier for loading the dataset
- `name`: displayed in the app
- `type`: specify Texture format:

value	associated format
0	Texture2d
1	TextureArray (KTX)

value	associated format
2	TextureArray (RAW Bytes)
3	~~Texture3D (DDS)~~
4	~~Texture3D (KTX)~~

- `folder`: directory in which the files are stored, relative to `assets/data`
- `layers`: Array containing the description of the texture file

Specific properties for TextureArrays

- `numLayers`: specifies how many textures are stored in the array
- `resX`: width of each texture in the array
- `resY`: height of each texture in the array
- `pixelFormat`: PixelFormat used by THREE.js to interpret the binary data (only needed for raw byte data)

value	associated pixel format
1024	RGB
1028	RedFormat

- layers should contain only a single texture

Example Data

Visible Human

- obtained from Visible Human project: National Library of Medicine

CT Head Raw TextureArray

- obtained from CodeProject article
- License: CPOL
- based on Three.js example:
  - Demo
  - Code

Generating KTX files

- Install Khronos Texture Tools from github:
- see documentation:
- for creating a layered texture with 10 images use:

```
    "C:\Program Files\KTX-Software\bin\toktx.exe" --t2 --encode etc1s --layers 10
output_texArray @files.txt
```

- creates a `output_texArray.ktx2` file containing the 10 images specified in `files.txt` in ktx2 format (switch `--t2`)
- encoding `etc1s` is mandatory, otherwise, the KTX Loader cannot read the file (`Unsupported vkFormat` or KTXLoader throws a transcoding error)
- **Remark**: file paths in `files.txt` must be relative to the path from which the command is executed, alternatively, the prefix @@ can be used to specify file paths relative to `files.txt`
- **Remark**: currently `KTXLoader2` relies on a WASM transcoder, which needs to be configured at runtime (?). As workaround, the relevant code has been copied to `assets/jsm/`
  See also:
  - THREE.js docs
  - github issue
- **Remark**: current version of KTX2Loader requires KTXTools Version **4.1** or higher (current release candidate: Download)
- **Remark**: There seem to be issues with the color space when using KTX textures. KTX uses linear sRGB color space, Three.js assumes sRGB color space. As a result, colors are way too dark when using KTX textures. Unfortunately, there is no way to specify the color space for the compressed texture. So, the only way is to convert colors in the shader. As utility functions for these operations seem to be broken in the current THREE.js version, manual conversion is necessary in the fragment shader:

```
// conversion from Linear color space to sRGB
vec4 fromLinear(vec4 linearRGB)
{
  bvec4 cutoff = lessThan(linearRGB, vec4(0.0031308));
  vec4 higher = vec4(1.055)*pow(linearRGB, vec4(1.0/2.4)) - vec4(0.055);
  vec4 lower = linearRGB * vec4(12.92);

  return mix(higher, lower, cutoff);
}

// conversion from Linear color space to sRGB
vec4 toLinear(vec4 sRGB)
{
  bvec4 cutoff = lessThan(sRGB, vec4(0.04045));
  vec4 higher = pow((sRGB + vec4(0.055))/vec4(1.055), vec4(2.4));
  vec4 lower = sRGB/vec4(12.92);

  return mix(higher, lower, cutoff);
}
```

- **Remark:** there is also a good texture viewer (including KTX textures), available at github: kopaka1822/ImageViewer @ github

- **Remark:** if the images contain an ICC profile, the KTX tool fails to create the TextureArray, as it cannot embed or covert ICC profile data. One workaround is to remove the ICC profile from the files. One tool which can be used for this is PNGCrush, using the command

```
pngcrush_1_8_11_w64.exe -ow -rem allb -reduce file.png
```

or in a batch file iterating over all pngs in a directory (Windows):

```
For /R %%i in (*.png) do pngcrush_1_8_11_w64.exe -ow -rem allb -reduce %%i
```

Sources for Additional Content

Additional Content	**Link**
Dresden Hochwasser (2D/3D)	Dresden Hochwasser (arcGis)
Dresden Themenstadtplan	Dresden Themenstadtplan (cardo.Map)
Historische Karten Europa 1500 - 2008	Digitaler Atlas zur Geschichte Europas
Altar von Ghent (van Eyck) [**nicht freigegeben !**]	Closer to van Eyck
Visible Human Head	National Library of Medicine
CT Head Raw	CodeProject article

Electron

- application can be run as electron app using the command `npm run electron`
- application can be packaged as electron app using the command `npm run electron-build`
- resulting electron files are stored in the `release` folder
- both commands start a production build before packaging
- **IMPORTANT:** Windows .exe files are limited to 2GB file size. As NSIS packages all resources into one large installer, this means that you have to be careful regarding the number of texture resources contained in the packaged installer. if the package is too large, the application is correctly built and copiend to `win-unpacked` directory, but the installer file is corrupted and the build process fails with an error like this:

```
  File: failed creating mmap of "D:\Projekte\ReFlex\reflex-layers\reflex-
layers\release\reflex-layers-0.9.0-x64.nsis.7z"
  Error in macro x64_app_files on macroline 1
  Error in macro compute_files_for_current_arch on macroline 7
  Error in macro extractEmbeddedAppPackage on macroline 8
  Error in macro installApplicationFiles on macroline 79
```

```
    !include: error in script: "installSection.nsh" on line 66
    Error in script "<stdin>" on line 189 -- aborting creation process
```

Keyboard Shortcuts

The app starts in full screen mode. The following shortcuts are available:

Shortcut	Description
`S`	Toggle Settings Panel
`Esc`	Close the application
`STRG` + `Shift` + `I`	open dev tools
`STRG` + `R`	reload app
`STRG` + `M`	minimize app

Interaction modes

Pixel Blending

- Blends the image based on the color value from the depth sensor
- Options:
  - **Stream Sensor Depth**: Determine whether raw depth image is used for blending or greyscale representation of filtered PointCloud (*Default:* **false**)
  - **Show Depth**: for debugging: show streamed depth image (*Default:* **false**)
  - **Interpolate colors**: smooth depth image fpr reducing artefacts (*Default:* **true**)
  - **Overrride Depth**: for debugging: manually set depth value (full screen) (*Default:* **false**)
  - **Min Depth Value**: greyscale value mapped to deepest point (*Default:* **0.0**)
  - **Zero Depth Value**: greyscale value mapped to zero plane point (*Default:* **0.5**)
  - **Apply Calibration**: transform depth image based on the translate/scale values derived from the coordinate mapping between sensor and interaction space (*Default:* **true**). If this is not set, depth image is stretched to fill the screen space

Magic Lens (Single or Multi-Touch)

- Displays a lens at the finger position containing the content based on the deformation at the fingertip
- In Single-touch mode, the first touch is used to determine the lens position, in multi-touch mode, a lens for every finger is displayed
- Options:
  - **Lens Size**: Scaling factor for the lens (*Default:* **1.0**)
  - **Lens Offset X**: move the lens from the fingertips in horizontal direction (*Default:* **0.0**)
  - **Lens Offset Y**: move the lens from the fingertips in vertical direction (*Default:* **0.0**)
  - **Show Lens UI**: show current layer at the border of the lens (*Default:* **true**)
  - **Show Layer UI**: show layers and current position of the lenses at the side of the screen (*Default:* **true**)

Layer Navigation

- go through layers by deforming the screen, complete layer is displayed based on the deepest finger position
- Options:
    - **Show Layer UI**: show layers and current position at the side of the screen (*Default:* **true**)

When using KTX-Arrays, **Lens Modes** offer an additional option to specify the lens mask. his can be customized to achieve effects such as a fisheye effect. Also, the border color of the lens can be specified.